

Name: Chinta Lakshmi Nivas Jaswanth

Module 3

1) Control Structures

Scenario 1: Apply 1% Discount on Loan Interest Rate for Customers Above 60

```
DECLARE
  CURSOR c_customers IS
    SELECT CustomerID
    FROM Customers
    WHERE Age > 60;
BEGIN
  FOR cust IN c_customers LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE CustomerID = cust.CustomerID;
    DBMS_OUTPUT.PUT_LINE(
      'Discount applied for Customer ID: ' || cust.CustomerID
    );
  END LOOP;

  COMMIT;
END;
/
```

Scenario 2: Set VIP Status for Customers with Balance Over \$10,000

```
DECLARE

    CURSOR c_customers IS
        SELECT CustomerID
        FROM Customers
        WHERE Balance > 10000;
BEGIN
    FOR cust IN c_customers LOOP
        UPDATE Customers
        SET IsVIP = 'TRUE'
        WHERE CustomerID = cust.CustomerID;
        DBMS_OUTPUT.PUT_LINE(
            'VIP status granted to Customer ID: ' || cust.CustomerID
        );
    END LOOP;

    COMMIT;
END;
/
```

Scenario 3: Send Reminders for Loans Due Within Next 30 Days

```
DECLARE

    CURSOR c_loans IS
        SELECT c.CustomerID,
               c.Name,
               l.LoanID,
               l.DueDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30;
BEGIN
```

```

FOR loan_rec IN c_loans LOOP
    DBMS_OUTPUT.PUT_LINE(
        'Reminder: Dear ' || loan_rec.Name ||
        ', your Loan ID ' || loan_rec.LoanID ||
        ' is due on ' || TO_CHAR(loan_rec.DueDate, 'DD-MON-YYYY')
    );
END LOOP;
END;
/

```

2) Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest for Savings Accounts

Stored Procedure: ProcessMonthlyInterest:

```

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');
```

END;

/

Execute Procedure:

```
BEGIN  
    ProcessMonthlyInterest;  
END;  
/
```

Scenario 2: Update Employee Bonus

Stored Procedure: UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    p_DepartmentID IN NUMBER,  
    p_BonusPercent IN NUMBER  
)  
IS  
BEGIN  
    UPDATE Employees  
    SET Salary = Salary + (Salary * p_BonusPercent / 100)  
    WHERE DepartmentID = p_DepartmentID;  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Bonus updated successfully.');
```

```
END;  
/
```

Execute Procedure:

BEGIN

 UpdateEmployeeBonus(101, 10);

END;

/

Scenario 3: Transfer Funds Between Accounts

Stored Procedure: TransferFunds

CREATE OR REPLACE PROCEDURE TransferFunds(

 p_FromAccount IN NUMBER,

 p_ToAccount IN NUMBER,

 p_Amount IN NUMBER

)

IS

 v_Balance NUMBER;

BEGIN

 SELECT Balance

 INTO v_Balance

 FROM Accounts

 WHERE AccountID = p_FromAccount;

 IF v_Balance >= p_Amount THEN

 UPDATE Accounts

 SET Balance = Balance - p_Amount

 WHERE AccountID = p_FromAccount;

 UPDATE Accounts

```
        SET Balance = Balance + p_Amount
    WHERE AccountID = p_ToAccount
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

ELSE

```
        DBMS_OUTPUT.PUT_LINE('Insufficient balance.');
```

END IF;

END;

/

Execute Procedure:

```
BEGIN
    TransferFunds(1001, 1002, 5000);
END;
```

/